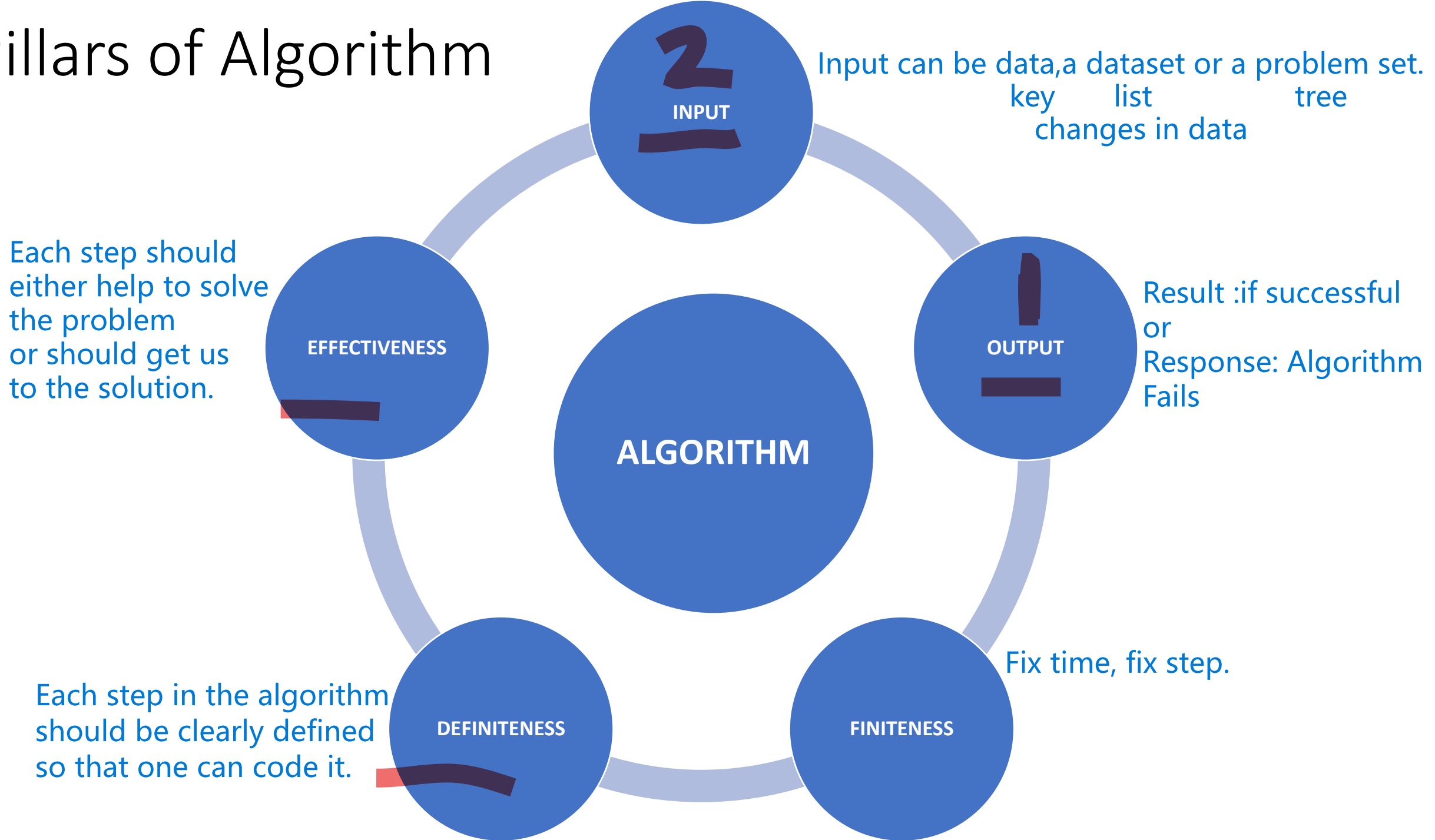


Algorithms

What is Algorithm ?

- Set of finite steps if followed will solve given problem in finite steps and finite time.

Pillars of Algorithm



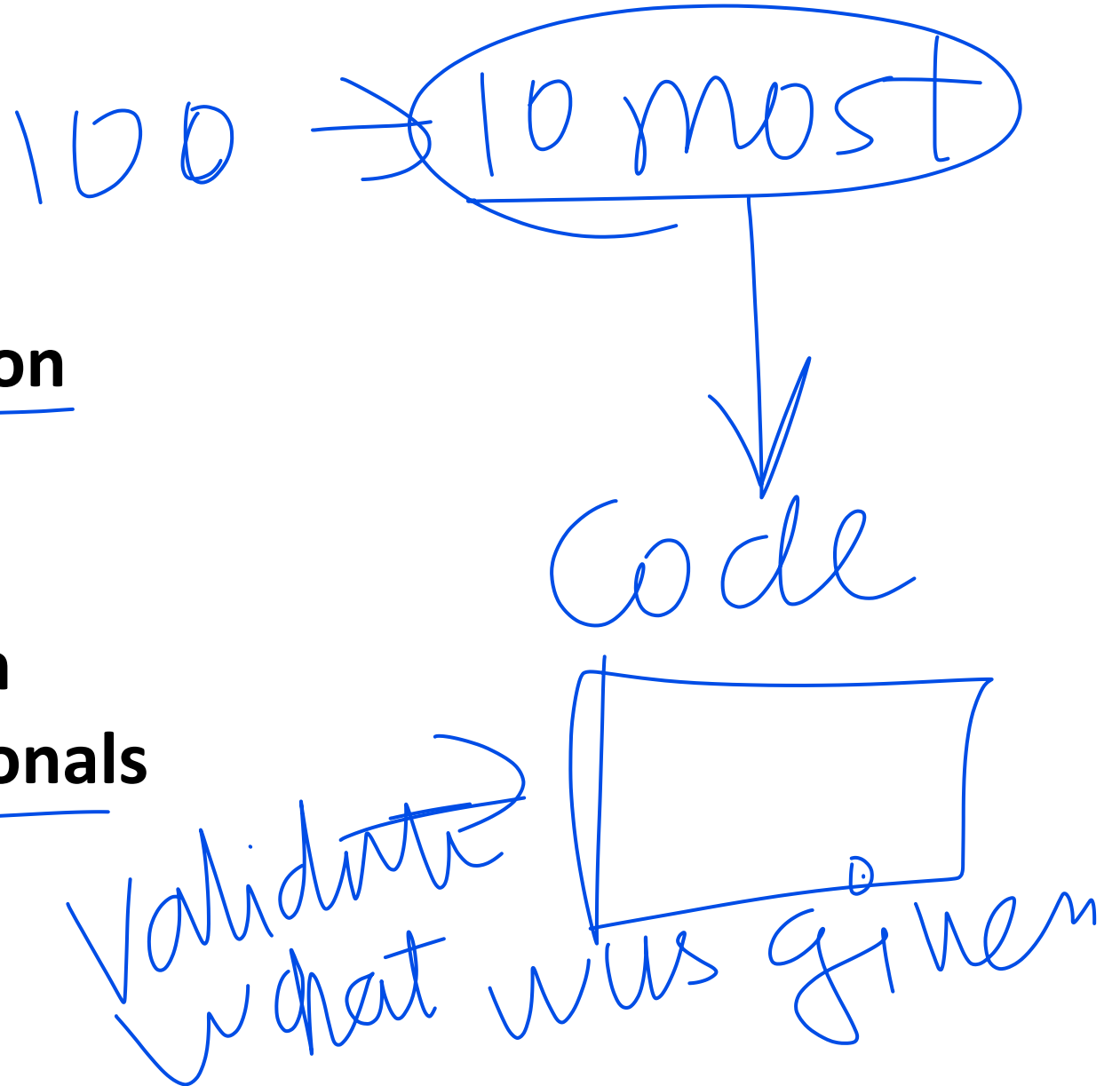
Analysis ?

- Why ?

- So one can predict **time and space** complexity and then **select the one.**

Analysis ?

- When ?
- **Priori** ✓
 - Done before implementation
 - Using pen paper
- **Posteriori** ✓
 - Done after implementation
 - Using software by professionals



Analysis ?

- What we get ?

- Time Complexity

$$T(P) = t_{\text{compile}}(P) + t_{\text{execute}}(P)$$

(Note: The above equation is crossed out with a large X in the original image. Below it, the terms are written as $t_{\text{compile}}(P)$ and $t_{\text{execute}}(P)$ with circled 1 and n respectively.)

- Space Complexity

$$S(P) = S_{\text{constant, var}}(P) + S_{\text{instructions}}(P)$$

Analysis ?

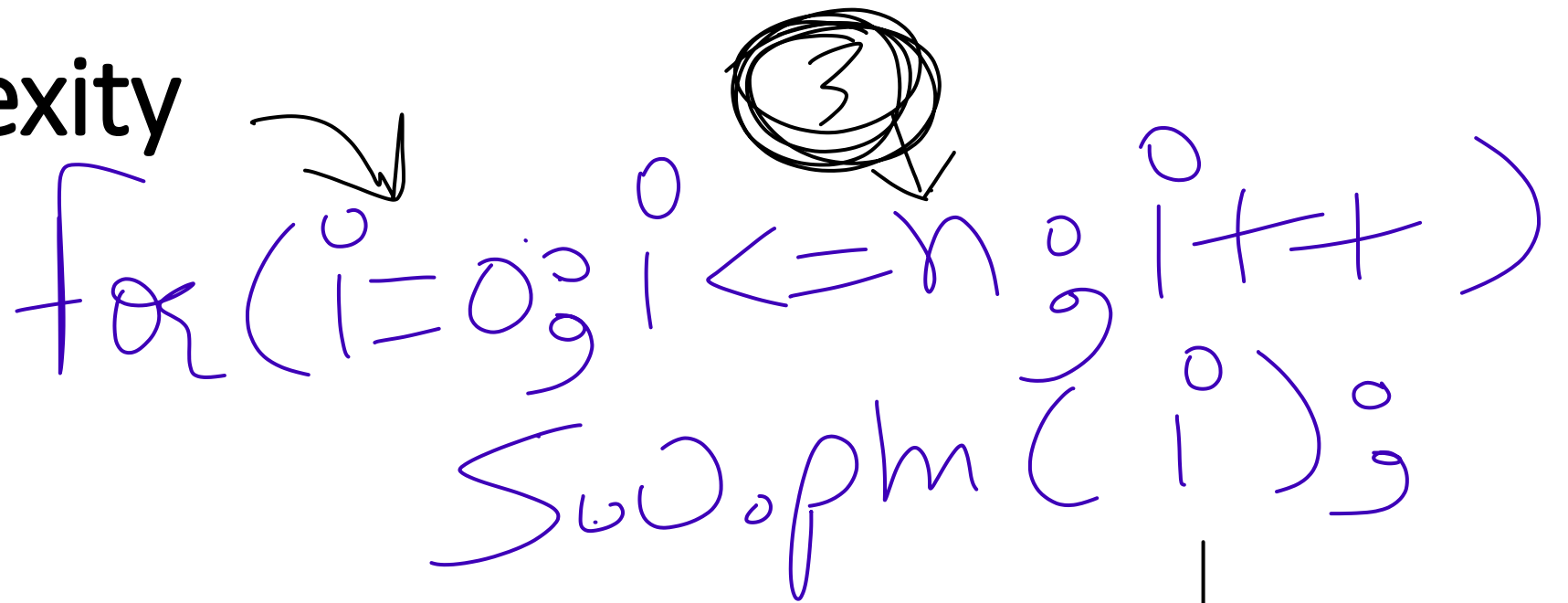
- **On What?**

- **Best Case** Minimum or no work?
- **Worst Case** For a problem size of n , amount of work would be a multiple of n .
- **Average Case** For problem size of n , near about $n/2$ amount of work.

Which one of these you will use and why?
worst as we get all needed details

Time Complexity

- Comments -0
- Operation----1
- Loop-----n+k


 $\text{for}(i=0; i \leq n; i++)$
 $\text{Swap}(i);$

$i=0 \rightarrow 1$

$i \leq n \rightarrow 0, 1, 2, 3, 4 \dots n+2$

$\text{Print} \rightarrow 0, 1, 2, 3 \dots n+1$

$i++ \rightarrow 1, 2, 3, 4 \dots n+1$

$\frac{3n+5}{\text{Steps}}$

Problem

- An algorithm takes 0.5 milli seconds for 100 inputs calculate time if input is increased to 500 and if complexities are

- Linear

$n \Rightarrow \text{steps}$

100 $\rightarrow n \rightarrow 100$ steps
500 $\rightarrow n \rightarrow 500$ steps

- Quadratic

n^2

- Cubic

n^3

- Log based

$\log n$

100 — 0.5 msec
500 — ?
 $(500 \times 0.5) / 100 = 2.5$ msec

Asymptotic Notation

- **Asymptotic Notation is used to describe the running time of an algorithm - how much time an algorithm takes with a given input, n .**

- **Algorithmic Common Runtimes**

- fastest to slowest are:
- constant: $\Theta(1)$
- logarithmic: $\Theta(\log N)$
- linear: $\Theta(N)$
- polynomial: $\Theta(N^2)$
- exponential: $\Theta(2^N)$
- factorial: $\Theta(N!)$

A recording complexity using math form
kind of unit for recording complexity.

Big Oh
Theta
Omega

Asymptotic Notation

- **Asymptotic Notation is used to describe the running time of an algorithm - how much time an algorithm takes with a given input, n .**
- **Algorithmic Common Runtimes**
- fastest to slowest are:
- constant: $\Theta(1)$
- logarithmic: $\Theta(\log N)$
- linear: $\Theta(N)$
- polynomial: $\Theta(N^2)$
- exponential: $\Theta(2^N)$
- factorial: $\Theta(N!)$

Big Oh:

targets worst case

easy and fast

-ve: poor efficiency due to Big O rules.

Theta:

targets $(\text{worst} + \text{best}) / 2$ ---> average of both

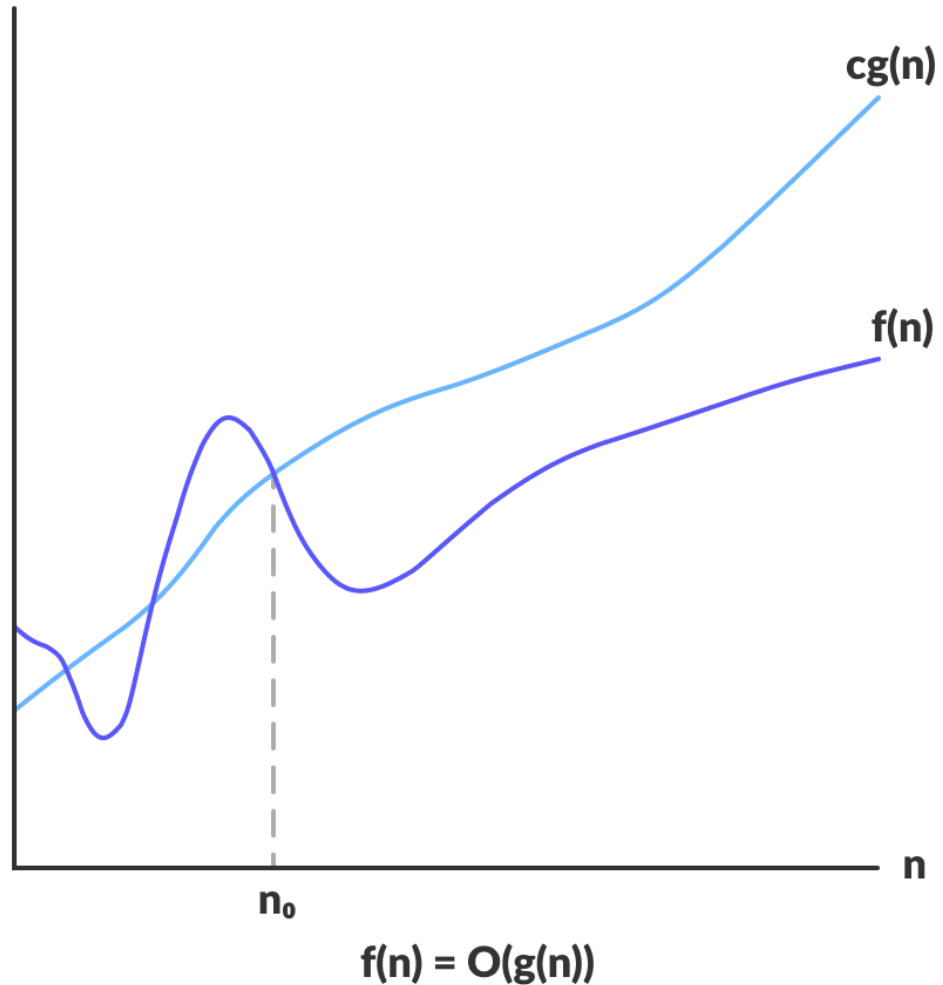
best accurate

-ve: slow as best and worst both need to be monitored.

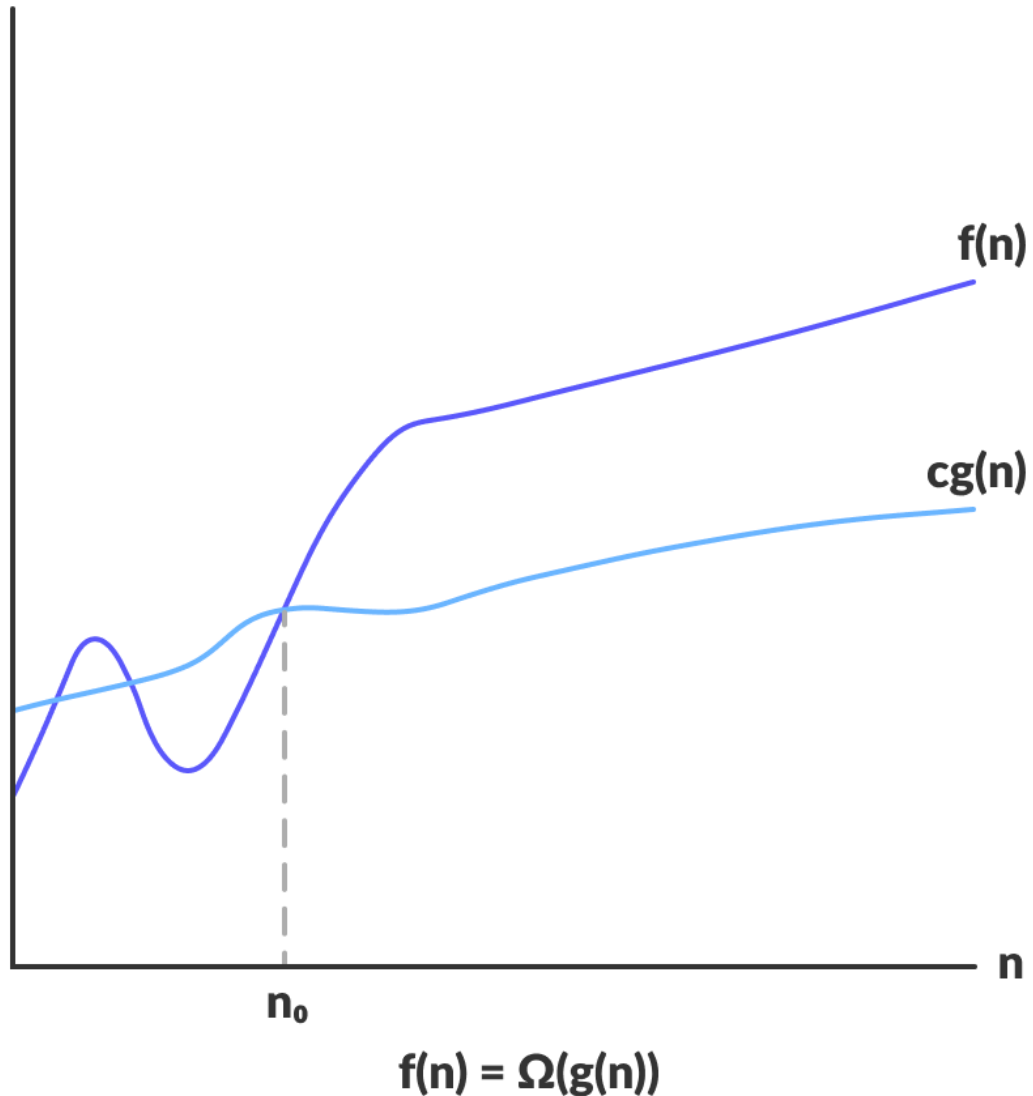
Omega:

targets best case

- **Big-O Notation (O-notation)**
- Big-O notation represents the upper bound of the running time of an algorithm. Thus, it gives the worst-case complexity of an algorithm.

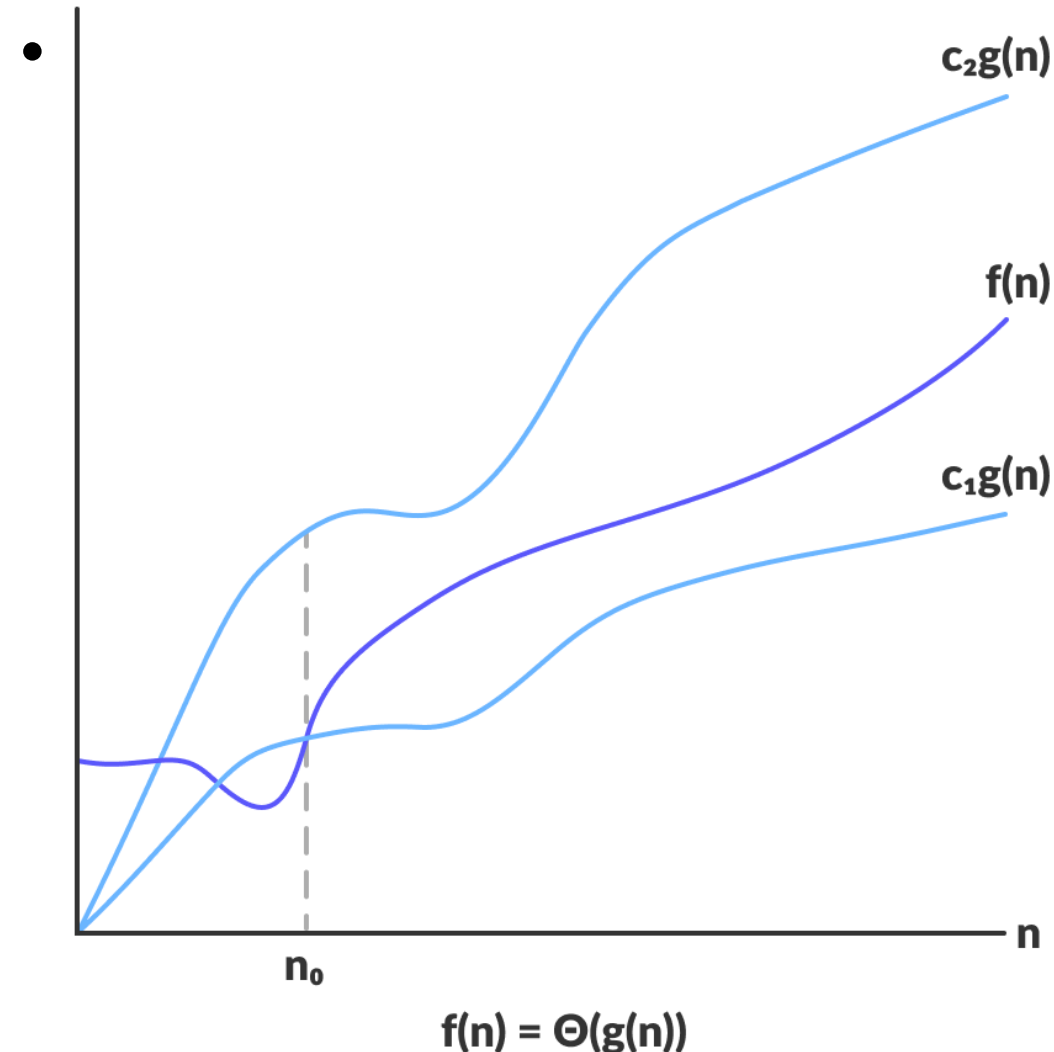


- **Omega Notation (Ω -notation)**
- Omega notation represents the lower bound of the running time of an algorithm. Thus, it provides the best case complexity of an algorithm.



- **Theta Notation (Θ -notation)**

- Theta notation encloses the function from above and below. Since it represents the upper and the lower bound of the running time of an algorithm, it is used for analyzing the average-case complexity of an algorithm.



• Big-O Notation (O-notation)

• Rules

- Constant has no value $O(100n) \Rightarrow O(n)$
- In addition use max(worst only)
- In nesting multiply and then use addition rule

$$\begin{array}{r} \cancel{O(n)} + \cancel{O(n^2)} + \cancel{O(n^3)} \\ \hline 1000 \quad 100 \quad 10 \\ O(n^3) \end{array}$$

$$O(n) + O(n+2) \Rightarrow \underline{O(n^2)} + \cancel{O(n)}$$

Calculate Complexity using Big OH

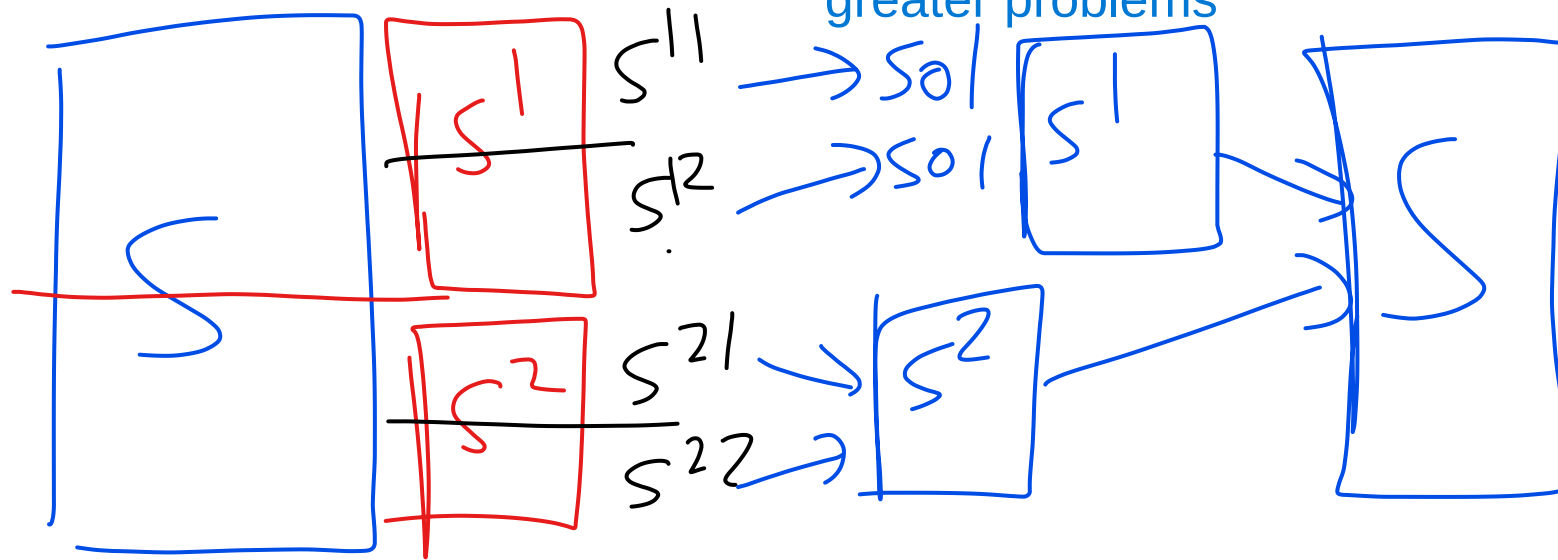
Types of Algorithms

1 Divide and conquer

- Merge sort
- Binary search

when problem size > what you can handle
div problem its half and keep doing it
either reached to size you can solve it
or unit(1)

then solve the sub problem and then substitute its answer to
greater problems

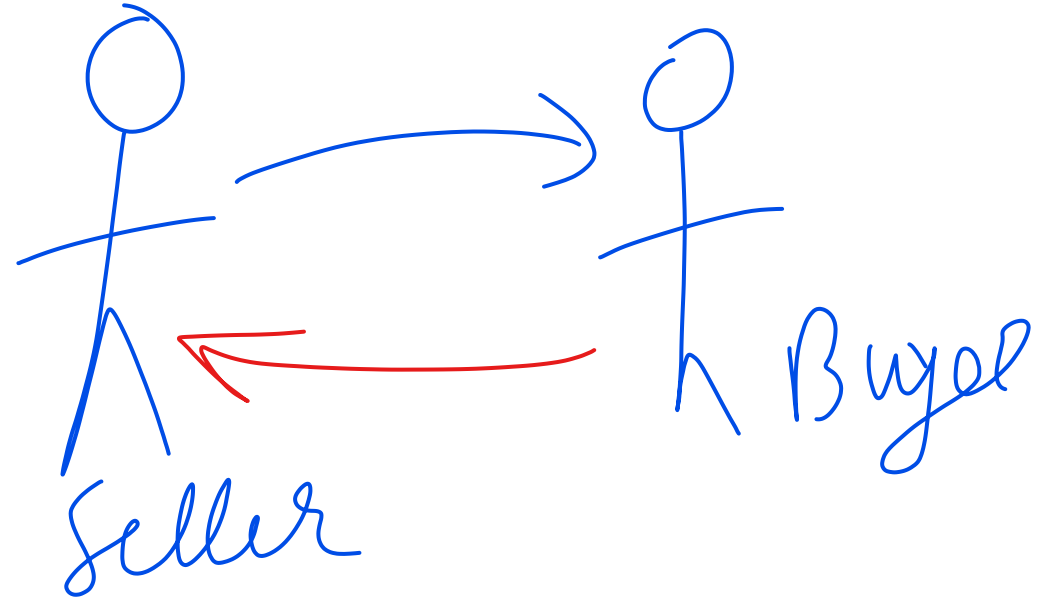


Types of Algorithms

2 Greedy Algorithm

- Talks About Optimization
- Optimization
 - Minimization-Cost
 - Maximization-Profit

Shortest Path First
MST

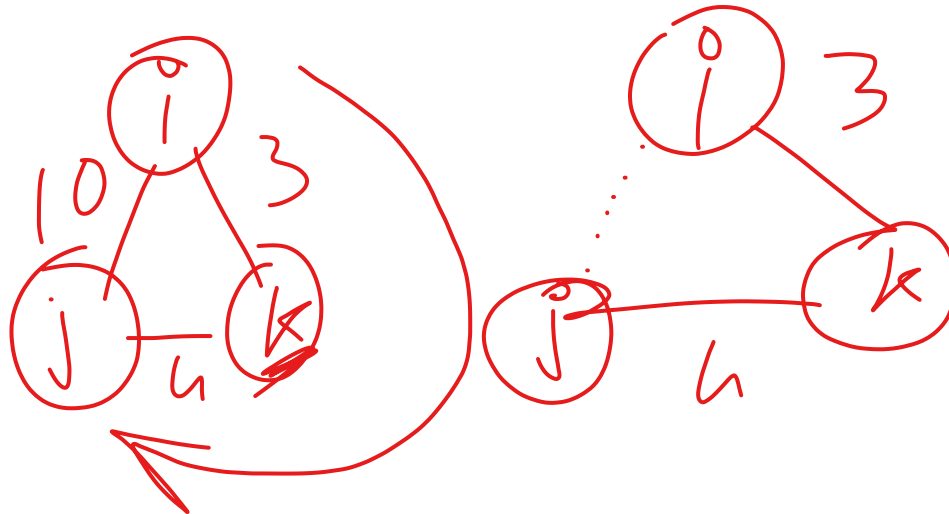


Types of Algorithms

3 Dynamic Programming

Share tracking / LIFE

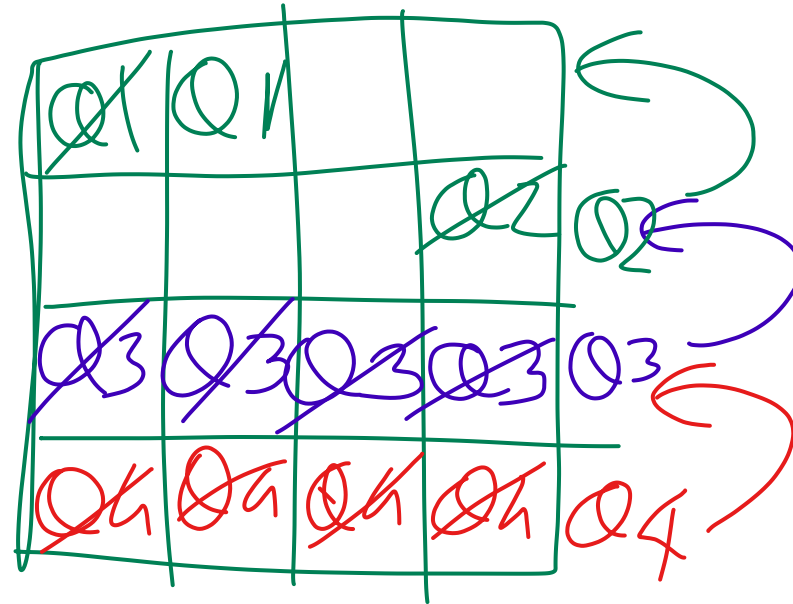
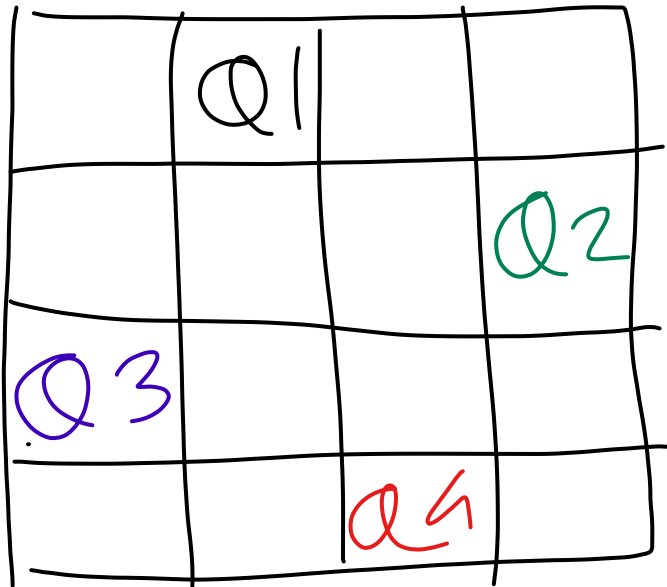
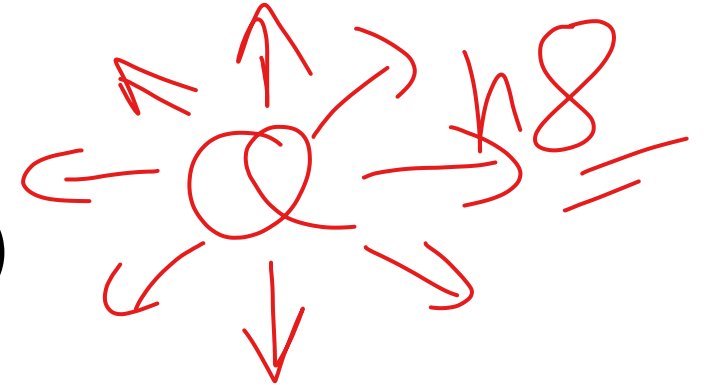
- Break problem in states, evaluate each state with formula recursively, select best in each step so finally get optimized answer.
- Example: All source shortest path (Floyd warshal's)



Types of Algorithms

4 Back Tracking

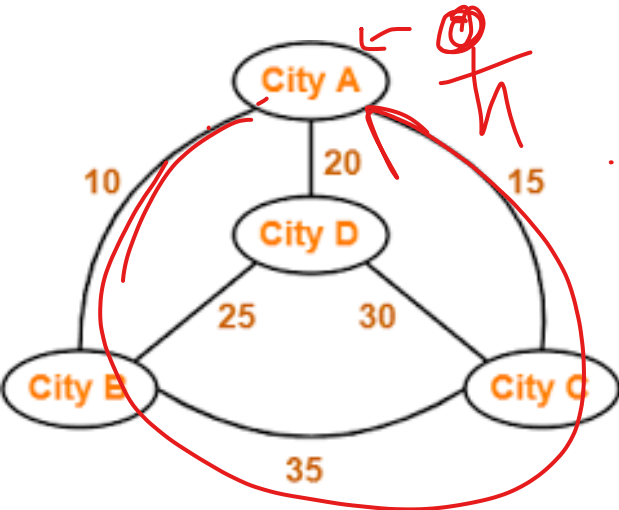
- Only solution needed . (May/May not be optimal)
- N-queens problem $n \times n - n$



Types of Algorithms

5 Branch n Bound

- All solutions ✓
- State Space tree(decision tree) ✓
- Example: Traveling salesman



Travelling Salesman Problem

Min Cost

45
 AB C
 75
 ABCD
 95
 35
 ADD
 65
 ABDC
 80

A
 10
 5
 50
 ACB
 75
 ACBD
 95
 20
 45
 ACD
 70
 ACDB
 80
 45
 AD
 50
 ADB
 80
 ADDB
 95
 ADC
 185
 ADCB
 95

Types of Algorithms

- **6 STOCHASTIC ALGORITHM**

- Stochastic refers to a variable process where the outcome involves some randomness and has some uncertainty. It is a mathematical term and is closely related to “*randomness*” and “*probabilistic*” and can be contrasted to the idea of “*deterministic*.”
- May produce better results

